Workflow Tests Without Rails

Limiting model tests to use only ActiveRecord doesn't seem like that big a win overall, since almost by definition a model test doesn't explore anything beyond the model.

Ideally, you'd also be able to isolate your workflow object specs, since those workflow objects are specifically created to be plain Ruby objects. Also, if you aren't careful, testing these actions can be particularly slow because you potentially create and save a lot of model objects.

You can often isolate these objects by using test doubles as replacements for your ActiveRecord objects. However, test doubles can be difficult as replacements for Rails associations (you want to model them as lists, but they aren't exactly lists). The two techniques can also work together, in that the code you would write to make it easier to isolate ActiveRecord also often makes the remaining code easier to use with test doubles.

Your existing CreatesProject tests can be isolated without much trouble, although there are rather a lot of dependencies. The header looks like this:

environment/01/spec/workflows/creates_project_spec.rb

```
require_relative "../active_record_spec_helper"
require "devise"
require "devise/orm/active_record"
require_relative "../../app/models/project"
require_relative "../../app/models/task"
require_relative "../../app/models/role"
require_relative "../../app/models/user"
require_relative "../../app/workflows/creates_project"
```

About that list of dependencies: it seems kind of long. Especially since four of those require statements—the two devise lines, the role, and the user—are there only to support the User class. You need to import the User class in part because you have a line in the action that assigns the incoming user objects to project.users. That line causes Rails to autoload the User class, which brings in Role and indirectly brings in Devise because the User class calls the devise method to set up authentication.

That seems like a lot of dependency being brought in for a single line. When you were using Rails in this test, Rails autoload prevented you from having to care about dependencies. Now that you're specifying them explicitly, you can see how interconnected this test is.